PySpark Optimization

Commands & Techniques

Make Every Spark Job Faster | Azure Data Engineer Edition

Partitioning | Caching | Joins | AQE | Delta Lake | Skew Fix | UDFs

Every command | Every config | Every anti-pattern — 2025 Edition

1. Diagnose Before You Optimize

Never tune blindly. Always profile first — understand the bottleneck, then apply the right fix.

Execution Plan Analysis

```
# View execution plans (always start here) df.explain() # simple physical plan df.explain("extended")
# logical + optimized + physical plans df.explain("cost") # with cost estimates df.explain("codegen")
# generated Java code (Tungsten) df.explain("formatted") # nicely formatted physical plan # Equivalent
SQL spark.sql("EXPLAIN SELECT * FROM orders WHERE status='active'") spark.sql("EXPLAIN EXTENDED SELECT
* FROM orders WHERE status='active'") spark.sql("EXPLAIN COST SELECT * FROM orders WHERE
status='active'")
```

Spark UI Metrics to Watch

Metric	What it reveals
`Stage Duration`	Which stage is the bottleneck
`Shuffle Read/Write`	How much data crosses the network (shuffle)
`Spill (Memory)`	Data that doesn't fit in executor memory -> slow disk I/O
`Spill (Disk)`	Actual bytes written to disk due to memory pressure
`GC Time`	JVM garbage collection — too high = memory pressure
`Task Skew`	Min vs Max task time — big gap means data skew
`Input Size`	How much data is actually being read (check pruning works)
`Output Size`	Useful to detect accidental cartesian products

Partition & Row Count Diagnostics

```
# Check partition count and skew df.rdd.getNumPartitions() # total partitions
df.rdd.glom().map(len).collect() # rows per partition list # Stats per partition (quick skew check)
df.rdd.glom().map(len).stats() # min/max/mean/stdev # Count rows, distinct values, nulls per column
df.count() df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns]).show()
df.select([F.countDistinct(c).alias(c) for c in df.columns]).show() # Data size estimate
spark.sql("ANALYZE TABLE catalog.schema.table COMPUTE STATISTICS") spark.sql("DESCRIBE EXTENDED
catalog.schema.table") # check sizeInBytes
```

2. Partitioning & Shuffle Optimization

Shuffle is the #1 performance killer in Spark. Minimising shuffle and right-sizing partitions is the single highest-impact optimization.

Core Partitioning Commands

```
# Reshuffle to n partitions (full shuffle — use before wide ops) df.repartition(200) # Partition by
column value (data with same key goes to same partition) df.repartition(200, "country")
df.repartition(200, "dept", "year") # multiple columns # Reduce partitions WITHOUT full shuffle (only
merge, never split) df.coalesce(10) # use after filter/limit to reduce file count # Recommended
shuffle partition formula # spark.sql.shuffle.partitions = 2x to 3x total executor cores # e.g. 10
workers x 4 cores = 40 cores -> set 80 to 120 partitions
spark.conf.set("spark.sql.shuffle.partitions", "100") # File partition size (how large each input
split is) spark.conf.set("spark.sql.files.maxPartitionBytes", str(128 * 1024 * 1024)) # 128 MB
spark.conf.set("spark.sql.files.openCostInBytes", str(4 * 1024 * 1024)) # 4 MB # Default RDD
parallelism spark.conf.set("spark.default.parallelism", "100")
```

Adaptive Query Execution (AQE) — Spark 3+

```
# Enable all AQE features (recommended for all Spark 3+ jobs)
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.minPartitionNum","1")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "134217728") # 128 MB
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5") # 5x median
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "268435456") # 256 MB
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", "true") # reduce shuffle read overhead
```

TIP AQE dynamically re-partitions after each shuffle stage based on actual data sizes. Enable it by default on all Spark 3+ jobs — it often gives 20-50% speedup for free.

Handling Data Skew

```
# 1. Identify skewed partition df.groupBy("join_key").count().orderBy(F.col("count").desc()).show(20)
# 2a. Salting technique (manual skew fix) SALT_BUCKETS = 10 df_large = df_large.withColumn("salt",
(F.rand() * SALT_BUCKETS).cast("int")) \ .withColumn("salted_key", F.concat_ws("_", F.col("key"),
F.col("salt"))) df_small = df_small.withColumn("salt", F.explode(F.array([F.lit(i) for i in
range(SALT_BUCKETS)]))) \ .withColumn("salted_key", F.concat_ws("_", F.col("key"), F.col("salt")))
result = df_large.join(df_small, "salted_key").drop("salt","salted_key") # 2b. AQE skew join
(automatic — just enable AQE above, no code change needed) # 2c. Split skewed key into separate jobs
skewed_keys = ["key_A", "key_B"] normal_keys = df.filter(~F.col("key").isin(skewed_keys)) skewed_part
= df.filter( F.col("key").isin(skewed_keys)) # process separately and union results
```

Reduce Shuffle — Query Design Patterns

```
# Filter EARLY before any join/groupBy df = df.filter(F.col("year") == 2024).filter(F.col("status") ==
"active") # Select only needed columns BEFORE joining df_orders =
spark.read.table("orders").select("id","customer_id","amount","date") df_customers =
spark.read.table("customers").select("id","name","region") result = df_orders.join(df_customers,
df_orders["customer_id"] == df_customers["id"]) # Avoid groupByKey on RDDs (shuffles all data) — use
reduceByKey instead # BAD: rdd.groupByKey().mapValues(sum) # GOOD: rdd.reduceByKey(lambda a,b: a+b) #
combines on map side first # Use mapPartitions instead of map for I/O-heavy per-row operations def
process_partition(iter): # open DB connection ONCE per partition, not per row conn =
get_db_connection() for row in iter: yield enrich(row, conn) df.rdd.mapPartitions(process_partition)
```

3. Join Optimization

Broadcast Join — Fastest Join Strategy

```
from pyspark.sql.functions import broadcast # Explicit broadcast hint (table < ~200 MB fits in executor memory) df_large.join(broadcast(df_small), on="product_id", how="left") # Auto-broadcast threshold (default 10 MB — increase for larger lookups) spark.conf.set("spark.sql.autoBroadcastJoinThreshold", str(50 * 1024 * 1024)) # 50 MB # Disable auto-broadcast (useful for debugging) spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1") # Check if broadcast was used df_large.join(broadcast(df_small), "id").explain() # Look for: BroadcastHashJoin in the physical plan
```

Join Strategy Hints (Spark 3+)

```
# Force specific join strategies with hints df1.hint("broadcast").join(df2, "key") # BroadcastHashJoin df1.hint("merge").join(df2, "key") # SortMergeJoin df1.hint("shuffle_hash").join(df2, "key") # ShuffleHashJoin df1.hint("shuffle_replicate_nl").join(df2,"key") # BroadcastNestedLoopJoin # SQL hints spark.sql(''' SELECT /*+ BROADCAST(small_tbl) */ * FROM large_tbl l JOIN small_tbl s ON l.id = s.id ''')
```

Join Type Selection Guide

Situation	Best Join Strategy
`One table < 50-200 MB`	broadcast() — no shuffle at all
`Both tables large, sorted by key`	SortMergeJoin — efficient for huge tables
`Check existence only (no cols needed)`	left_semi join — half the data movement
`Find rows with no match`	left_anti join — efficient existence check
`Multiple small dimension tables`	Chain broadcasts: join(broadcast(d1)).join(broadcast(d2))
`Joining on nullable key columns`	Filter nulls first: df.filter(F.col('key').isNotNull())
`Skewed join key`	Enable AQE skew join or use salting technique

Avoid Common Join Pitfalls

```
# Problem: ambiguous column names after join # BAD result = df1.join(df2, df1["id"] == df2["id"]) result.select("id") # AnalysisException: ambiguous reference # GOOD — rename before joining df2_renamed = df2.withColumnRenamed("id", "cust_id") result = df1.join(df2_renamed, df1["id"] == df2_renamed["cust_id"]) # GOOD — use alias result = df1.alias("a").join(df2.alias("b"), F.col("a.id") == F.col("b.id")) result.select(F.col("a.id"), F.col("b.name")) # Problem: joining on partition key causes unnecessary shuffle # GOOD — repartition both DFs on same key first df1_p = df1.repartition(200, "customer_id") df2_p = df2.repartition(200, "customer_id") result = df1_p.join(df2_p, "customer_id") # shuffle already done
```

4. Caching & Persistence

Cache Commands

```
from pyspark import StorageLevel # Quick cache (MEMORY_AND_DISK — safe default) df.cache() df.count() # IMPORTANT: materialise the cache with an action # Explicit storage levels df.persist(StorageLevel.MEMORY_ONLY) # fastest; recomputes if OOM df.persist(StorageLevel.MEMORY_AND_DISK) # spills to disk (default cache) df.persist(StorageLevel.DISK_ONLY) # always on disk; slower but safe
```

```
df.persist(StorageLevel.MEMORY_ONLY_2) # 2 replicas in memory
df.persist(StorageLevel.MEMORY_AND_DISK_2) # 2 replicas, spills to disk
df.persist(StorageLevel.OFF_HEAP) # Tungsten off-heap (no GC pressure) # Release cache df.unpersist()
df.unpersist(blocking=True) # wait until freed # Table-level cache spark.catalog.cacheTable("orders")
spark.catalog.uncacheTable("orders") spark.catalog.isCached("orders") # check if cached
spark.catalog.clearCache() # clear ALL cached tables
```

Checkpoint — Truncate Long Lineage

```
# Set checkpoint directory (ADLS or DBFS)
sc.setCheckpointDir("abfss://container@storage.dfs.core.windows.net/checkpoints/") # Checkpoint saves
data to storage and cuts the lineage DAG df.checkpoint() # reliable (saves to ADLS)
df.localCheckpoint() # faster (saves locally, lost on failure) # When to checkpoint: # - After 50+
chained transformations (long lineage = slow planning) # - Before an iterative ML loop # - After a
very expensive aggregation you don't want to recompute
```

Storage Level	Use When
`MEMORY_ONLY`	Data fits in RAM; re-compute is cheap if evicted
`MEMORY_AND_DISK`	Default safe choice; spills to disk if needed
`DISK_ONLY`	Data is large; reading disk faster than recomputing
`MEMORY_ONLY_2`	Need fault tolerance (executor failure recovery)
`OFF_HEAP`	Avoiding GC pressure on very large cached datasets
`checkpoint()`	Breaking long lineage chains (50+ transformations)
`localCheckpoint()`	Same as checkpoint but faster — no storage guarantee

RULE Cache ONLY if you reuse a DataFrame 2+ times in the same job. Unnecessary caching wastes memory and slows down other executors.

AVOID Never cache a streaming DataFrame. Never cache right before a write — the cache is wasted.

5. UDF Optimization — Replace & Vectorize

Python UDFs are the most common cause of slow PySpark jobs. Each row crosses the Python-JVM boundary individually. Always prefer built-in functions, then Pandas UDFs, then Python UDFs as a last resort.

Use Built-in Functions First (Zero Overhead)

```
# SLOW: Python UDF from pyspark.sql.functions import udf from pyspark.sql.types import StringType
@udf(StringType()) def grade_udf(score): if score >= 90: return "A" elif score >= 75: return "B" else:
return "C" df.withColumn("grade", grade_udf(F.col("score"))) # slow: row-by-row Python # FAST:
Built-in F.when (stays in JVM, no serialization) df.withColumn("grade", F.when(F.col("score") >= 90,
"A") .when(F.col("score") >= 75, "B") .otherwise("C"))
```

Pandas UDF (Vectorized) — 10-100x Faster than Python UDF

```
from pyspark.sql.functions import pandas_udf import pandas as pd # Series -> Series (element-wise
transformation) @pandas_udf("double") def adjust_salary(s: pd.Series) -> pd.Series: return s * 1.1 +
500 df.withColumn("new_salary", adjust_salary(F.col("salary"))) # Series -> Scalar (aggregate)
@pandas_udf("double") def weighted_avg(val: pd.Series, weight: pd.Series) -> float: return (val *
weight).sum() / weight.sum() df.groupBy("dept").agg(weighted_avg(F.col("score"), F.col("weight"))) #
Iterator of Series -> Iterator (efficient batch transform with setup cost) from typing import Iterator
@pandas_udf("string") def batch_transform(iterator: Iterator[pd.Series]) -> Iterator[pd.Series]: model
= load_model() # load ONCE per partition, not per row for s in iterator: yield model.predict(s)
df.withColumn("prediction", batch_transform(F.col("features")))
```

Register & Reuse UDFs

```
# Register for Spark SQL reuse spark.udf.register("grade_fn", lambda x: "A" if x>=90 else "B",
"string") spark.sql("SELECT id, grade_fn(score) AS grade FROM students") # KryoSerializer speeds up
Python object serialization spark.conf.set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer") spark.conf.set("spark.kryo.registrationRequired",
"false")
```

TIP Apache Arrow must be installed for Pandas UDFs: pip install pyarrow. Set spark.sql.execution.arrow.pyspark.enabled=true.

6. I/O & File Format Optimization

File Format Choice

Format	Best For
`Delta`	Default choice on Databricks — ACID, time travel, Z-Order, auto-compaction
`Parquet`	Columnar, compressed — fast reads with column pruning & predicate pushdown
`ORC`	Similar to Parquet, better for Hive — use when interoperating with Hive
`Avro`	Row-based — best for Kafka/streaming where you write every field every row
`CSV/JSON`	Only for human-readable exchange or external system ingestion — avoid in prod

Predicate & Projection Pushdown

```
# Column pruning — only read columns you need (Parquet/Delta will skip others) df =
spark.read.parquet("path/").select("id","name","amount") # reads 3 cols only # Predicate pushdown —
filter on partition columns to skip entire files df = spark.read.parquet("path/year=2024/") # read
only 2024 data df = spark.read.parquet("path/").filter(F.col("year") == 2024) # Spark auto-pushes this
# Enable pushdown (on by default) spark.conf.set("spark.sql.parquet.filterPushdown", "true")
spark.conf.set("spark.sql.orc.filterPushdown", "true")
spark.conf.set("spark.sql.parquet.aggregatePushdown", "true") # push agg to file scan # Verify
pushdown happened — look for "PushedFilters" in explain df.filter(F.col("status") ==
"active").explain() # Physical Plan will show: PushedFilters: [IsNotNull(status),
EqualTo(status,active)]
```

Partitioned Write Strategy

```
# Partition by low-cardinality columns (year, month, region, status) df.write.mode("overwrite") \
.partitionBy("year","month") \ .parquet("path/") # Control file count (avoid small file problem)
df.repartition(10).write.mode("overwrite").parquet("path/") # 10 files
df.coalesce(1).write.mode("overwrite").csv("path/") # single file (small data only) # Target ~128-256
MB per output file # rows_per_file = target_bytes / avg_row_size_bytes # num_files = total_rows /
rows_per_file # df.repartition(num_files).write... # Dynamic partition overwrite — overwrite only
touched partitions (not the whole table) spark.conf.set("spark.sql.sources.partitionOverwriteMode",
"dynamic") df.write.mode("overwrite").partitionBy("year","month").parquet("path/")
```

Compression

```
# Parquet compression (snappy=default, zstd=better ratio, uncompressed=fastest decode)
df.write.option("compression","snappy").parquet("path/")
df.write.option("compression","zstd").parquet("path/") # best compression ratio
df.write.option("compression","gzip").parquet("path/") # highest ratio, slow
df.write.option("compression","uncompressed").parquet("path/") # fastest scan, large files # Global
codec setting spark.conf.set("spark.sql.parquet.compression.codec", "snappy")
spark.conf.set("spark.io.compression.codec", "snappy")
```

Small File Problem — Fix & Prevention

```
# Delta OPTIMIZE — compact small files into ~1 GB files spark.sql("OPTIMIZE catalog.schema.table")
spark.sql("OPTIMIZE catalog.schema.table WHERE year = 2024") # Z-Order — co-locate related data
(multi-column filter optimization) spark.sql("OPTIMIZE catalog.schema.table ZORDER BY (customer_id,
order_date)") # Auto-optimize on Databricks Delta
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true") # smart write sizing
spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true") # background compaction # Set per
table spark.sql("ALTER TABLE orders SET TBLPROPERTIES ('delta.autoOptimize.optimizeWrite'='true')") #
Liquid Clustering (Databricks — replaces Z-Order and static partitioning) spark.sql("ALTER TABLE
orders CLUSTER BY (customer_id, order_date)") spark.sql("OPTIMIZE orders") # incremental clustering
```

7. Memory & Executor Configuration

Key Memory Configurations

Config Property	Recommended Setting / Notes
`spark.executor.memory`	4g–16g depending on cluster. Start with 4g.
`spark.executor.memoryOverhead`	10–15% of executor memory (for off-heap, Python processes)
`spark.driver.memory`	2g–8g. Increase if collecting large DataFrames.
`spark.driver.maxResultSize`	2g (default 1g). Increase for large collect() calls.
`spark.memory.fraction`	0.6 (default). Fraction for execution + storage.
`spark.memory.storageFraction`	0.5 (default). Fraction of spark.memory.fraction for storage.
`spark.executor.cores`	2–4 recommended. Higher = more threads per executor, more GC.
`spark.sql.execution.arrow.pyspark.enabled`	true — enables Arrow for Pandas UDFs & toPandas()
`spark.python.worker.memory`	512m — memory for each Python worker process
`spark.executor.extraJavaOptions`	-XX:+UseG1GC -XX:InitiatingHeapOccupancyPercent=35

Dynamic Allocation

```
# Auto-scale executors based on load spark.conf.set("spark.dynamicAllocation.enabled", "true")
spark.conf.set("spark.dynamicAllocation.minExecutors", "2")
spark.conf.set("spark.dynamicAllocation.maxExecutors", "20")
spark.conf.set("spark.dynamicAllocation.initialExecutors", "4")
spark.conf.set("spark.dynamicAllocation.executorIdleTimeout", "60s")
spark.conf.set("spark.dynamicAllocation.schedulerBacklogTimeout","1s")
spark.conf.set("spark.dynamicAllocation.sustainedSchedulerBacklogTimeout","5s")
spark.conf.set("spark.shuffle.service.enabled", "true") # required for dynamic alloc
```

Spill & OOM Prevention

```
# Reduce shuffle partitions (less memory per partition) spark.conf.set("spark.sql.shuffle.partitions",
"500") # more, smaller partitions # Increase spill threshold
spark.conf.set("spark.sql.files.maxPartitionBytes", str(64 * 1024 * 1024)) # 64 MB partitions #
Off-heap memory (bypasses JVM GC) spark.conf.set("spark.memory.offHeap.enabled", "true")
spark.conf.set("spark.memory.offHeap.size", "2g") # If groupBy/join causes OOM — increase executor
memory overhead spark.conf.set("spark.executor.memoryOverhead", "2048") # MB # Kryo serializer —
reduces memory footprint of RDD/cached data spark.conf.set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
spark.conf.set("spark.kryoserializer.buffer.max","512m")
```

8. Spark SQL & Catalyst Optimizer Tricks

Force / Hint the Optimizer

```
# Repartition hint in SQL spark.sql("SELECT /*+ REPARTITION(200) */ * FROM orders") spark.sql("SELECT
/*+ REPARTITION(200, dept) */ * FROM orders") spark.sql("SELECT /*+ COALESCE(5) */ * FROM orders") #
Join hints in SQL spark.sql(''' SELECT /*+ BROADCAST(d) */ o.id, o.amount, d.name FROM orders o JOIN
departments d ON o.dept_id = d.id ''') spark.sql("SELECT /*+ MERGE(o, d) */ ... -- force
SortMergeJoin") spark.sql("SELECT /*+ SHUFFLE_HASH(o) */ ... -- force ShuffleHashJoin") # Statistics
hints spark.sql("ANALYZE TABLE orders COMPUTE STATISTICS") spark.sql("ANALYZE TABLE orders COMPUTE
```

```
STATISTICS FOR ALL COLUMNS") spark.sql("ANALYZE TABLE orders COMPUTE STATISTICS FOR COLUMNS amount, status, dept_id")
```

Subquery & CTE Optimization

```
# CTEs are generally more readable AND let Catalyst optimize better result = spark.sql(''' WITH active_orders AS ( SELECT * FROM orders WHERE status = 'active' AND year = 2024 ), customer_totals AS ( SELECT customer_id, SUM(amount) AS total FROM active_orders GROUP BY customer_id ) SELECT c.name, ct.total FROM customer_totals ct JOIN customers c ON ct.customer_id = c.id WHERE ct.total > 1000 ORDER BY ct.total DESC ''') # Use spark.sql() for complex multi-step logic # Break into multiple temp views for readability df_filtered.createOrReplaceTempView("filtered_orders") df_enriched.createOrReplaceTempView("enriched_orders") spark.sql("SELECT ... FROM filtered_orders JOIN enriched_orders ON ...")
```

Column Pruning & Filter Pushdown Verification

```
# Verify column pruning is working df = spark.read.parquet("path/") df.select("id","amount").explain() # Expect: PhysicalRDD or FileScan with only 2 columns listed # Verify predicate pushdown df.filter(F.col("status") == "active").explain() # Expect: PushedFilters: [IsNotNull(status), EqualTo(status,active)] # Force statistics collection for better Catalyst decisions spark.sql("ANALYZE TABLE orders COMPUTE STATISTICS FOR ALL COLUMNS") spark.conf.set("spark.sql.cbo.enabled", "true") # cost-based optimizer spark.conf.set("spark.sql.cbo.joinReorder.enabled","true") # auto-reorder joins
```

9. Delta Lake Optimization Commands

OPTIMIZE & Z-Order

```
-- Compact small files into ~1 GB target files OPTIMIZE catalog.schema.orders; OPTIMIZE
catalog.schema.orders WHERE year = 2024; -- Z-Order: co-locate rows that will be queried together --
Put your most common filter/join columns here (max 4) OPTIMIZE catalog.schema.orders ZORDER BY
(customer_id); OPTIMIZE catalog.schema.orders ZORDER BY (customer_id, order_date); -- Liquid
Clustering (Databricks Runtime 13.3+ — replaces Z-Order) ALTER TABLE orders CLUSTER BY (customer_id,
order_date); OPTIMIZE orders; -- incremental, idempotent
```

VACUUM & Retention

```
-- Remove old data files no longer referenced (default: retain 7 days) VACUUM catalog.schema.orders;
VACUUM catalog.schema.orders RETAIN 168 HOURS; -- 7 days VACUUM catalog.schema.orders RETAIN 0 HOURS
DRY RUN; -- preview files to delete -- Shorten retention for cost savings (be careful with streaming!)
ALTER TABLE orders SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 2 days');
```

Auto-Optimize & Table Properties

```
-- Enable auto-optimize per table (Databricks) ALTER TABLE orders SET TBLPROPERTIES (
'delta.autoOptimize.optimizeWrite' = 'true', -- smart output file sizing
'delta.autoOptimize.autoCompact' = 'true' -- background compaction ); -- Enable Change Data Feed ALTER
TABLE orders SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true'); -- Set target file size ALTER
TABLE orders SET TBLPROPERTIES ('delta.targetFileSize' = '134217728'); -- 128 MB -- Statistics
collection (helps Catalyst optimizer) ALTER TABLE orders SET TBLPROPERTIES
('delta.dataSkippingNumIndexedCols' = '10');
```

Data Skipping & File Pruning

```
# Delta automatically collects min/max stats on first 32 columns # Queries with filters on those
columns skip irrelevant files # Check how many files are skipped spark.sql(''' DESCRIBE DETAIL
catalog.schema.orders ''').select("numFiles","sizeInBytes").show() # Check stats collection
spark.sql("DESCRIBE EXTENDED orders").filter("col_name = 'Statistics'").show() # Force stats refresh
spark.sql("ANALYZE TABLE orders COMPUTE DELTA STATISTICS") # Databricks # Partition pruning still
applies on top of data skipping df = spark.read.table("orders").filter( (F.col("year") == 2024) & #
partition pruning (F.col("customer_id") > 1000) # data skipping on min/max stats )
```

10. Essential spark.conf Settings — Quick Reference

Set these at the SparkSession level or in your cluster configuration.

Config Key	Recommended Value / Notes
`spark.sql.shuffle.partitions`	2-3x total executor cores (default: 200)
`spark.sql.adaptive.enabled`	true — enable AQE (Spark 3+)
`spark.sql.adaptive.coalescePartitions.enabled`	true — auto-merge small post-shuffle partitions
`spark.sql.adaptive.skewJoin.enabled`	true — auto-fix skewed joins
`spark.sql.adaptive.advisoryPartitionSizeInBytes`	134217728 (128 MB) — target partition size after coalesce
`spark.sql.autoBroadcastJoinThreshold`	52428800 (50 MB) — auto-broadcast tables under this size
`spark.sql.files.maxPartitionBytes`	134217728 (128 MB) — max bytes per input file partition
`spark.sql.parquet.filterPushdown`	true — push filters into Parquet scan
`spark.sql.parquet.aggregatePushdown`	true — push aggregates into Parquet scan

Config Key	Recommended Value / Notes
`spark.sql.cbo.enabled`	true — cost-based optimizer
`spark.sql.cbo.joinReorder.enabled`	true — auto reorder joins based on stats
`spark.sql.execution.arrow.pyspark.enabled`	true — Arrow for Pandas UDFs & toPandas()
`spark.serializer`	org.apache.spark.serializer.KryoSerializer
`spark.kryoserializer.buffer.max`	512m — increase for large objects
`spark.sql.sources.partitionOverwriteMode`	dynamic — overwrite only affected partitions
`spark.databricks.delta.optimizeWrite.enabled`	true — smart write sizing (Databricks only)
`spark.databricks.delta.autoCompact.enabled`	true — background compaction (Databricks only)
`spark.sql.adaptive.localShuffleReader.enabled`	true — reduce shuffle overhead
`spark.memory.offHeap.enabled`	true — use off-heap to reduce GC pressure
`spark.memory.offHeap.size`	2g — off-heap size per executor
`spark.dynamicAllocation.enabled`	true — auto-scale executors
`spark.shuffle.service.enabled`	true — required for dynamic allocation
`spark.sql.broadcastTimeout`	600 — seconds to wait for broadcast (default 300)
`spark.network.timeout`	800s — increase for long-running stages
`spark.executor.heartbeatInterval`	60s — heartbeat to driver

OPTIMIZATION CHEATSHEET

```
# ■■ Step 1: Always diagnose first ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
df.explain("extended") # read the physical plan df.rdd.glom().map(len).stats() # detect partition skew
# ■■ Step 2: Enable AQE (free wins) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true") # ■■ Step 3: Right-size shuffle
partitions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
spark.conf.set("spark.sql.shuffle.partitions", "100") # = 2-3x total cores # ■■ Step 4: Filter &
select EARLY ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ df =
df.filter(F.col("year")==2024).select("id","name","amount") # before any join # ■■ Step 5: Broadcast
small tables ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ from pyspark.sql.functions
import broadcast result = df_large.join(broadcast(df_small), "key", "left") # ■■ Step 6: Repartition
before wide ops ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ df.repartition(200,
"customer_id").groupBy("customer_id").agg(F.sum("amount")) # ■■ Step 7: Coalesce before write
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
df.coalesce(10).write.mode("overwrite").parquet("path/") # avoid small files # ■■ Step 8: Cache only
reused DataFrames ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ df_base =
df.filter(...).select(...).cache() df_base.count() # materialise result1 =
df_base.groupBy("dept").agg(...) result2 = df_base.join(lookup, "key") df_base.unpersist() # ■■ Step
9: Use built-ins over UDFs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ # SLOW:
df.withColumn("x", my_udf(F.col("val"))) # FAST: df.withColumn("x",
F.when(F.col("val")>0,"pos").otherwise("neg")) # ■■ Step 10: Fix skew with salting
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ SALT = 10 df_big =
df_big.withColumn("salt",(F.rand()*SALT).cast("int")) \ .withColumn("skey",
F.concat_ws("_","key","salt")) df_sm = df_sm.withColumn("salt", F.explode(F.array([F.lit(i) for i in
range(SALT)]))) \ .withColumn("skey", F.concat_ws("_","key","salt"))
df_big.join(df_sm,"skey").drop("salt","skey") # ■■ Step 11: Delta maintenance
```

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ spark.sql("OPTIMIZE catalog.schema.table ZORDER BY (customer_id, order_date)") spark.sql("VACUUM catalog.schema.table RETAIN 168 HOURS") # ■■ Step 12: Pandas UDF instead of Python UDF ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ from pyspark.sql.functions import pandas_udf @pandas_udf("double") def fast_transform(s: pd.Series) -> pd.Series: return s * 1.1 df.withColumn("new_val", fast_transform(F.col("val")))
```

Apply these optimizations in order — diagnose first, then fix the biggest bottleneck. A 10x speedup is often possible by enabling AQE, adding a single broadcast(), and right-sizing partitions alone.